Quantum Shield MLOps Project - Updated Complete Study Guide

Project Overview

Quantum Shield Neural Intrusion Hunter is an end-to-end MLOps project for network intrusion detection that trains a machine learning model, serves it through a FastAPI backend, deploys it in containers, exposes it through a frontend dashboard, and monitors it with Prometheus and Grafana.[cite:1] The project now includes a Kubernetes-based deployment using K3s on AWS EC2, while the frontend remains hosted on Netlify and updates automatically from the GitHub `frontend` branch.[cite:1]

The system predicts whether a network session is safe or malicious using a trained Random Forest model and returns an attack probability, a binary attack decision, a confidence level, and a message for the user interface.[cite:1] The project demonstrates a full practical MLOps workflow: training, packaging, deployment, observability, cloud hosting, and frontend integration.[cite:1]

Problem Statement

The goal of the project is to detect suspicious or malicious network sessions from structured session-level features such as packet size, protocol type, failed logins, unusual access time, IP reputation score, and browser type.[cite:1] Each input row represents one network session, and the target prediction is whether that session is an attack or not.[cite:1]

This problem is important because cybersecurity systems often need fast automated decisions on tabular event data, and the project simulates how such a model can move from experimentation into a production-style deployment pipeline.[cite:1]

Dataset and Prediction Task

The dataset contains network session features including `networkpacketsize`, `protocoltype`, `loginattempts`, `sessionduration`, `failedlogins`, `unusualtimeaccess`, `ipreputationscore`, and `browsertype`.[cite:1] The model predicts `isattack` and also returns `attackprobability`, `confidence`, and a human-readable message.[cite:1]

Feature	Type	Meaning
networkpacketsize	Numeric	Packet size in bytes [cite:1]
protocoltype	Categorical	Network protocol such as TCP, UDP, ICMP [cite:1]
loginattempts	Numeric	Number of login attempts [cite:1]
sessionduration	Numeric	Session duration in seconds [cite:1]
failedlogins	Numeric	Number of failed logins [cite:1]

Feature	Type	Meaning
unusualtimeaccess	Binary	Whether access happened at an unusual time [cite:1]
ipreputationscore	Numeric	Reputation score of source IP from 0 to 10 [cite:1]
browsertype	Categorical	Browser used in the session [cite:1]

Machine Learning Layer

The model used in the project is a Random Forest Classifier built with scikit-learn.[cite:1] Random Forest was chosen because it works well on tabular data, is robust to noise, and reduces overfitting by combining multiple decision trees.[cite:1]

Categorical fields were encoded using Label Encoding so the model could process non-numeric values such as protocol type and browser type.[cite:1] The trained model and encoder were stored as `model.pkl` and `encoder.pkl`, allowing the API to load them instantly during inference without retraining.[cite:1]

Backend API

The backend is built with FastAPI and exposes REST endpoints such as `/predict`, `/health`, and `/metrics`.[cite:1] The `/predict` endpoint accepts the network session features, applies the saved encoder, loads the trained model, performs inference, and returns the prediction response in JSON format.[cite:1]

The backend also includes API key authentication using the `X-API-Key` header, structured logging, error handling, and Prometheus metrics instrumentation.[cite:1] This makes the API both production-oriented and observable.[cite:1]

Docker Layer

Before Kubernetes was added, the FastAPI service was packaged into a Docker image and deployed directly on EC2.[cite:1] A typical Docker run command exposed host port 8001 to container port 8000, which made the API available publicly at `http://EC2_PUBLIC_IP:8001`. [cite:1]

Docker remains important in the updated setup because Kubernetes still runs the same container image from Docker Hub.[cite:1] In other words, Kubernetes did not replace Docker images; it replaced the old style of manually running containers with `docker run` and managing them one by one.[cite:1]

What Kubernetes Is

Kubernetes is a container orchestration system that manages how containers are deployed, restarted, exposed, and scaled across infrastructure.[cite:21][cite:17] Instead of manually starting containers and wiring ports every time, Kubernetes uses declarative YAML manifests that define the desired state of the application, and the cluster continuously works to maintain that state.[cite:17][cite:21]

In this project, Kubernetes was added using **K3s**, which is a lightweight Kubernetes distribution well suited for small EC2 machines.[cite:6] K3s is useful because a full Kubernetes cluster can be heavy for a student project, while K3s provides the same core concepts—pods, deployments, services, namespaces, health probes, and cluster management—with lower resource usage.[cite:6]

Why Kubernetes Was Added

Kubernetes was added to make the deployment more production-like and easier to manage over time.[cite:17] [cite:21] It provides automatic restart of failed containers, declarative deployment definitions, clean separation of application resources, health checks, and standard service exposure patterns.[cite:17] [cite:21]

For this project specifically, Kubernetes improves the system in these ways:

- The API now runs as a managed Deployment instead of an ad hoc Docker container.[cite:17]
- The service is exposed through a Kubernetes Service instead of a raw Docker port mapping.[cite:21]
- Readiness and liveness probes ensure the app is considered healthy only when `/health` responds correctly.[cite:17]
- Future additions such as scaling, rolling updates, and in-cluster monitoring become easier.[cite:21]

AWS EC2 Setup Evolution

Originally, the project used an Ubuntu EC2 instance to run Docker containers directly, and the backend image was pulled from Docker Hub by GitHub Actions over SSH.[cite:1] Later, the EC2 instance was resized from its smaller initial resource profile to a more practical state with approximately 2 GiB RAM and 20 GiB disk, making it capable of running K3s plus the application and monitoring tools.[cite:1]

Disk cleanup was also performed to free container image space before the Kubernetes installation, which was important because Kubernetes and container runtimes require additional storage for images, pods, and cluster components.[cite:1]

Kubernetes Installation and Cluster State

A single-node K3s cluster was installed on the EC2 instance.[cite:6] After increasing resources and restarting the environment, the cluster reached a healthy state with the single node marked `Ready`, allowing application workloads to be scheduled successfully.[cite:6]

A dedicated Kubernetes namespace named `quantum-shield` was created to isolate the project resources from the default and system namespaces.[cite:21] This improves organization and makes it easier to manage the project independently of Kubernetes internal components.[cite:21]

Kubernetes Resources Created

The main Kubernetes resources added for the API were a **Deployment** and a **Service** in the `quantum-shield` namespace.[cite:17] [cite:21]

Deployment

The Deployment named `quantum-shield-api` runs one replica of the Docker image `shriyasabnis/mlops:latest` and exposes container port 8000.[cite:17] It also defines resource requests and limits, along with readiness and liveness probes pointing to `/health` on port 8000.[cite:17]

This means Kubernetes continuously ensures that one healthy pod for the API is available. If the pod crashes, Kubernetes recreates it automatically.[cite:21]

Service

The Service named `quantum-shield-api-svc` was created as a `NodePort` service.[cite:21] It maps cluster traffic to the API pod and exposes the application externally on port `30080`, while internally forwarding requests to port `8000` in the pod.[cite:21]

This made the API accessible from outside the EC2 machine at:

- `http://EC2_PUBLIC_IP:30080/health`
- `http://EC2_PUBLIC_IP:30080/predict`

[cite:21]

Difference Between Old and New Deployment

Aspect	Old Docker setup	New Kubernetes setup
Runtime	Manual Docker container on EC2 [cite:1]	Pod managed by K3s Deployment [cite:17]
Public API port	8001 on EC2 [cite:1]	30080 via NodePort [cite:21]
Restart behavior	Manual or CI/CD-driven restart [cite:1]	Automatic pod reconciliation by Kubernetes [cite:21]
Health checking	Application health endpoint existed [cite:1]	Kubernetes readiness/liveness probes use `/health` [cite:17]
Resource grouping	Plain Docker container names [cite:1]	Namespace, Deployment, Service, Pod [cite:21]

Current Runtime Architecture

The updated architecture now works like this:

1. The frontend is hosted on Netlify and served over HTTPS.[cite:1]
2. The frontend sends requests to `/api/predict` from browser JavaScript.[cite:1]

3. Netlify uses a redirect/proxy rule to forward `/api/*` requests to the EC2 backend endpoint.[cite:1]
4. The EC2 backend endpoint now points to the Kubernetes NodePort `30080` instead of the old Docker port `8001`.[cite:1]
5. The Kubernetes Service forwards traffic to the API pod running the FastAPI model server on port `8000`.[cite:21]
6. The API loads the saved model and encoder, predicts the result, and returns JSON to the frontend.[cite:1]
7. Prometheus scrapes the `/metrics` endpoint, and Grafana visualizes those metrics.[cite:1]

Netlify and GitHub Branch Integration

Originally, the frontend was deployed by manually uploading the frontend folder to Netlify.[cite:1] Later, the setup was improved by pushing the frontend code to GitHub and connecting Netlify to the `frontend` branch of the `Shriya-Sabnis/MLops` repository so deployments happen automatically when changes are pushed to that branch.[cite:1]

This means frontend deployment is now Git-based:

- Update frontend files locally.
- Commit changes to the `frontend` branch.
- Push to GitHub.
- Netlify detects the push and automatically publishes the updated frontend.[cite:1]

The `_redirects` rule was updated so `/api` now points to the Kubernetes NodePort endpoint rather than the old Docker port, allowing the frontend to continue working without major JavaScript changes.[cite:1]

Prometheus Setup

Prometheus was used to monitor the FastAPI application by scraping its `/metrics` endpoint.[cite:1] In the original setup, Prometheus scraped `localhost:8001` because the API was exposed directly through Docker on that port.[cite:1]

After the Kubernetes migration, the Prometheus configuration was updated so the `quantum-shield-api` scrape target points to the EC2 private IP and NodePort `30080`, which forwards requests to the API pod in K3s.[cite:1] Prometheus itself continues to listen on port `9090` and can also scrape its own metrics through `localhost:9090`.[cite:100]

A simplified version of the updated scrape config is:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
```

```
      - targets: ["localhost:9090"]

  - job_name: "quantum-shield-api"
    scrape_interval: 10s
    metrics_path: /metrics
    static_configs:
      - targets: ["172.31.26.240:30080"]
```

This configuration allows Prometheus to gather current runtime metrics from the Kubernetes-hosted API.[cite:100] [cite:94]

Grafana Setup

Grafana was used as the visualization layer for Prometheus metrics.[cite:1] After the EC2 restart, Grafana could be started again from the existing Docker container, but dashboards would only show data after Prometheus was recreated and the data source was fixed.[cite:1]

The Prometheus data source in Grafana was configured to use the Prometheus endpoint exposed on the EC2 host at the private IP and port `9090`, which allowed Grafana to query Prometheus successfully.[cite:97] Once the data source returned "Successfully queried the Prometheus API," Grafana was correctly connected to Prometheus.[cite:97]

If older dashboards showed no charts after restart, the usual reasons were:

- The data source was not connected correctly.[cite:102]
- The queried metric names no longer matched the available metrics.[cite:102]
- There had not yet been enough recent traffic to generate fresh application metrics. [cite:102]

Why Grafana Dashboards May Show No Data

A working Grafana connection does not automatically mean every panel will show data. [cite:102] A panel shows data only if its query matches a metric currently available in Prometheus and the selected time range includes points for that metric.[cite:102] [cite:110]

In this project, once Prometheus and Grafana were reconnected, any dashboard panel using an outdated metric name or a range with no recent traffic could show "No data" even though the monitoring stack itself was healthy.[cite:102] [cite:110] Sending fresh `/predict` requests and checking available metrics through Prometheus or Grafana Explore is the correct way to validate which metrics should be used in dashboard panels.[cite:102]

Security Groups and Public Access

To expose the new Kubernetes API externally, the EC2 security group needed an inbound TCP rule for NodePort `30080`; otherwise the site would not be reachable from the browser even if the pod and service were healthy.[cite:57] Similarly, port `3000` is used for Grafana and `9090` for Prometheus when those dashboards are accessed from outside the instance.[cite:83] [cite:84]

This means the project's public access now depends on these main ports:

Port	Purpose
30080	Kubernetes NodePort for FastAPI API [cite:21]
3000	Grafana UI [cite:83]
9090	Prometheus UI [cite:100]
22	SSH access to EC2 [cite:1]

CI/CD Status After Kubernetes

The original CI/CD pipeline used GitHub Actions to build the Docker image, push it to Docker Hub, SSH into EC2, and run or restart the Docker container directly.[cite:1] After the Kubernetes migration, that old pipeline still exists conceptually, but the deployment model has changed because the live API now runs through a Kubernetes Deployment rather than a plain Docker container.[cite:1]

At the moment, the frontend deployment is GitHub-to-Netlify automated through the `frontend` branch, while the backend image is still based on Docker Hub and manually applied in Kubernetes using manifests.[cite:1] A future improvement would be to update GitHub Actions so it performs `kubectl apply` or a rollout restart instead of `docker run` on EC2.[cite:21]

What Was Accomplished During This Work Session

The main tasks completed during this Kubernetes extension were:

- Verified the EC2 environment, running containers, memory, and disk state.[cite:1]
- Cleaned disk space to prepare for Kubernetes installation.[cite:1]
- Installed K3s as a lightweight Kubernetes cluster on the EC2 instance.[cite:6]
- Resolved initial cluster instability by increasing storage and working with a more suitable runtime state on the instance.[cite:6] [cite:1]
- Created a dedicated `quantum-shield` namespace.[cite:21]
- Created and applied Deployment and Service manifests for the API.[cite:17] [cite:21]
- Exposed the API publicly using NodePort `30080` and verified `/health` and `/predict` externally.[cite:21]
- Updated the Netlify proxy to send `/api` traffic to the Kubernetes endpoint instead of the old Docker port.[cite:1]
- Confirmed the frontend works end-to-end with the Kubernetes backend.[cite:1]
- Restarted Grafana, recreated Prometheus, and updated monitoring targets for the new backend address.[cite:1] [cite:100]

Current End-to-End Flow

The system now runs as follows:

1. A user opens the Netlify frontend in the browser.[cite:1]
2. The frontend sends a request to `/api/predict`.[cite:1]
3. Netlify proxies the request to the EC2 public IP on NodePort `30080`.[cite:1]
4. The Kubernetes NodePort Service forwards traffic to the `quantum-shield-api` pod on port `8000`.[cite:21]
5. The FastAPI app loads `model.pkl` and `encoder.pkl`, validates input, runs inference, and returns the prediction JSON.[cite:1]
6. Prometheus scrapes `/metrics` from the same API through the NodePort target.[cite:1]
7. Grafana reads Prometheus data and visualizes the metrics in dashboards.[cite:1]

Kubernetes Concepts Used in This Project

Concept	Meaning in general	How it was used here
Cluster	A Kubernetes-managed environment for running workloads [cite:21]	A single-node K3s cluster on EC2 [cite:6]
Node	A machine inside the cluster [cite:21]	The EC2 instance itself [cite:6]
Namespace	Logical separation of resources [cite:21]	`quantum-shield` namespace for project isolation [cite:21]
Pod	Smallest deployable unit in Kubernetes [cite:21]	One pod running `shriyasabnis/mlops:latest` [cite:17]
Deployment	Manages pod lifecycle and desired replicas [cite:17]	Ensures one healthy API pod stays running [cite:17]
Service	Stable network access to pods [cite:21]	NodePort service exposing the API on 30080 [cite:21]
Readiness probe	Checks if pod is ready to receive traffic [cite:17]	`/health` endpoint on port 8000 [cite:17]
Liveness probe	Checks if pod should be restarted [cite:17]	`/health` endpoint on port 8000 [cite:17]

Key Files and Components in the Final Project

File / Component	Role
`mlmodel/trainmodel.py`	Trains the Random Forest model [cite:1]
`model.pkl`	Saved trained model [cite:1]
`encoder.pkl`	Saved label encoder [cite:1]
`app/main.py`	FastAPI app entry point with routes and Prometheus instrumentation [cite:1]

File / Component	Role
`model.py`	Loads model and performs prediction logic [cite:1]
`schemas.py`	Pydantic request/response models [cite:1]
`auth.py`	API key validation [cite:1]
`logger.py`	Logging configuration [cite:1]
`Dockerfile`	Builds backend image [cite:1]
`frontend/index.html`	Browser UI for live predictions and charts [cite:1]
`_redirects` / redirects	Netlify proxy config to backend [cite:1]
`quantum-shield-api.yaml`	Kubernetes Deployment + Service manifest for the API [cite:17][cite:21]
`~/prometheus/prometheus.yml`	Prometheus scrape configuration updated for K3s NodePort [cite:94] [cite:100]

Practical Explanation for Viva or Interview

A clear way to explain the final system is:

"Quantum Shield is a network intrusion detection MLOps project. A Random Forest model was trained on tabular session data and saved as a pickle artifact. A FastAPI backend loads the model and exposes prediction, health, and metrics endpoints. Initially the backend was deployed as a Docker container on AWS EC2, but later it was migrated to Kubernetes using K3s. The API now runs as a Kubernetes Deployment and is exposed through a NodePort Service on port 30080. The frontend is hosted on Netlify and auto-deploys from the GitHub `frontend` branch, while using a redirect rule to proxy browser requests to the Kubernetes backend. Prometheus scrapes the API metrics endpoint, and Grafana visualizes those metrics through dashboards."[cite:1][cite:17][cite:21]

Future Improvements

The current project is already fully functional, but several realistic next improvements remain possible:

- Move Prometheus and Grafana fully inside Kubernetes instead of running them as separate Docker containers.[cite:21]
- Replace manual backend manifest application with GitHub Actions `kubectl apply` or rollout automation.[cite:21]
- Add Horizontal Pod Autoscaler once CPU or custom metrics are available.[cite:21]
- Add Ingress and HTTPS for the backend instead of using a raw NodePort.[cite:21]
- Add persistent storage and saved Grafana dashboards for a more production-like observability setup.[cite:97]

Final Summary

The project now demonstrates a complete MLOps pipeline with machine learning training, containerization, backend API deployment, Kubernetes orchestration, frontend hosting, monitoring, and Git-based frontend auto-deployment.[cite:1] [cite:21] The most important architectural evolution was the move from a manually run Docker backend on EC2 to a Kubernetes-managed backend on K3s, while preserving the same model, API contract, and frontend behavior.[cite:1] [cite:17] [cite:21]

Anyone reading this document should understand both the original project and the updated Kubernetes-enabled deployment: what each layer does, why Kubernetes was introduced, how Prometheus and Grafana fit in, how Netlify and GitHub are integrated, and how the complete system currently runs in production-style form on AWS EC2.[cite:1] [cite:21]